



Projet Tutoré (PTUT)

SAE 3-01 : Projet d'analyse multivariée et reporting

Algorithme de Colonie de Fourmis (ACO)

Rapport de projet

Réalisé par :

Pap Moctar GAYE
Elhadj Abdoulaye GANGUE
Ibrahima TOURE

Encadré par :

Pr. Paul-Marie GROLLEMUND

Table des matières

Introduction	2
2. Présentation du problème étudié	3
3. Principe de l'algorithme de colonie de fourmis	3
4. Architecture du projet, choix techniques et implémentation	5
5. Comportement des fourmis	6
6. Algorithme global de la simulation	6
7. Algorithme inspiré du comportement des fourmis	7
8. Expérimentations et résultats	9
9. Analyse critique	9
10. Conclusion générale	10
11. Annexes	11

1. Introduction

Ce rapport d'étude s'inscrit dans le cadre du projet tutoré (PTUT) *SAE3-01 : Projet d'analyse multivariée et reporting*. La réalisation de ce travail s'est appuyée sur les recommandations et suggestions formulées lors de nos différents rendez-vous avec notre éminent professeur référent, **M. Grollemund**. Notre démarche méthodologique a consisté à aborder la problématique de manière progressive, en partant d'un cadre simple avant d'introduire progressivement des éléments de complexité. Cette approche incrémentale nous a permis de mieux comprendre les mécanismes étudiés et d'atteindre l'objectif final de cette étude.

Les algorithmes inspirés de la nature occupent aujourd'hui une place importante en informatique pour la résolution de problèmes complexes. Ils reposent sur l'observation de comportements simples présents dans la nature, lesquels, lorsqu'ils sont combinés, permettent de faire émerger des solutions efficaces. Parmi ces méthodes, les algorithmes de colonies de fourmis, connus sous le nom d'*Ant Colony Optimization* (ACO), initialement proposés par **Marco Dorigo**, sont largement utilisés pour résoudre des problèmes de recherche de chemin optimal, tels que le **problème du voyageur de commerce**.

Dans ce projet, nous avons étudié et simulé un algorithme de colonie de fourmis appliqué à un environnement discret. L'objectif est de déterminer un chemin reliant un point de départ, appelé le *nid*, à un point d'arrivée, appelé la *nourriture*, tout en tenant compte de la présence d'obstacles et de zones difficiles à traverser. Cet algorithme sera ensuite illustré dans un cadre plus concret, correspondant à la recherche du plus court chemin entre une salle de classe donnée et le restaurant universitaire.

2. Présentation du problème étudié

Le problème étudié consiste à trouver un chemin reliant le nid à la nourriture dans une grille discrète. Cette grille représente l'environnement dans lequel évoluent les fourmis, c'est-à-dire le cadre dans lequel se déroulent nos simulations.

Afin de rendre notre paradigme de programmation plus réaliste et plus intéressant, nous avons choisi de définir certaines cases de la grille comme des obstacles. Ces cases ne peuvent en aucun cas être traversées. D'autres cases sont définies comme des zones difficiles : elles restent franchissables, mais leur traversée implique un coût plus élevé. Ce choix permet de proposer une modélisation plus proche des contraintes observées dans la nature.

Le chemin recherché doit donc éviter les obstacles et, dans la mesure du possible, limiter le passage par les zones coûteuses. Chaque fourmi ne dispose que d'une information locale : elle ne connaît pas le chemin optimal et ne perçoit que les cases voisines immédiates. Son déplacement s'effectue ainsi étape par étape.

La solution finale n'est pas calculée de manière directe. Elle émerge progressivement grâce aux déplacements répétés des fourmis, à leurs interactions indirectes via l'environnement, et à l'accumulation d'informations au fil des itérations.

3. Principe de l'algorithme de colonie de fourmis

L'[algorithme de colonie de fourmis](#) est une méthode d'optimisation inspirée du comportement réel des fourmis dans la nature. Les fourmis explorent leur environnement afin de trouver des sources de nourriture. Lorsqu'une fourmi découvre une source de nourriture, elle retourne vers le nid en déposant une substance chimique appelée [phéromone](#) sur le sol.

Les autres fourmis ont tendance à suivre les chemins où la quantité de phéromone est la plus élevée. Cette quantité est calculée à partir de formules mathématiques qui modélisent le dépôt et l'évaporation des phéromones. Les chemins les plus courts ou les moins coûteux sont renforcés plus rapidement, car ils sont empruntés plus fréquemment. Avec le temps, la majorité des fourmis converge vers un même chemin.

Dépôt de phéromones

Lorsqu'une fourmi k emprunte un chemin reliant le nœud i au nœud j , elle dépose une quantité de phéromone définie par la relation suivante :

$$\Delta\tau_{i,j}^k = \frac{1}{L_k} \quad (1)$$

où :

- $\Delta\tau_{i,j}^k$ représente la quantité de phéromone déposée par la fourmi k sur l'arête reliant le nœud i au nœud j ;
- i, j désignent deux nœuds consécutifs du chemin parcouru ;
- k correspond à l'indice de la fourmi ;
- L_k est la longueur totale du chemin trouvé par la fourmi k .

Cette formulation traduit le fait que plus un chemin est court, plus la quantité de phéromone déposée est importante, ce qui favorise sa sélection par les autres fourmis.

Mise à jour globale des phéromones

La mise à jour de la quantité de phéromone sur chaque arête reliant les nœuds i et j est réalisée à chaque itération en combinant deux mécanismes : l'évaporation (qui diminue progressivement les traces) et le dépôt (qui renforce les chemins empruntés). Elle est modélisée par la relation suivante :

$$\tilde{\tau}_{i,j} = (1 - \rho) \tau_{i,j} + \sum_{k=1}^m \Delta\tau_{i,j}^k \quad (2)$$

où :

- $\tilde{\tau}_{i,j}$ représente la quantité de phéromone mise à jour sur l'arête reliant les nœuds i et j après évaporation et dépôt ;
- $\tau_{i,j}$ désigne la quantité de phéromone présente sur cette arête avant la mise à jour ;
- $\rho \in]0, 1[$ est le taux d'évaporation des phéromones, c'est-à-dire la proportion de phéromone perdue à chaque itération ;
- m correspond au nombre total de fourmis participant à la simulation ;
- $\Delta\tau_{i,j}^k$ est la quantité de phéromone déposée par la fourmi k sur l'arête (i, j) .

Cette équation permet d'éviter que des chemins non optimaux restent favorisés trop longtemps, tout en renforçant progressivement les trajets les plus pertinents au fur et à mesure des itérations.

Dans ce projet, le chemin optimal n'est pas nécessairement le plus court en termes de

distance. Il résulte d'un compromis entre :

- la longueur du chemin parcouru ;
- le coût cumulé des zones difficiles traversées.

La solution recherchée consiste donc à minimiser un coût global, intégrant à la fois la distance et les contraintes imposées par l'environnement.

4. Architecture du projet, choix techniques et implémentation

Afin de simplifier la gestion et la compréhension du projet, l'implémentation repose sur une approche de programmation modulaire. Le programme est découpé en plusieurs fichiers indépendants, chacun étant responsable d'une fonctionnalité précise : la définition des paramètres globaux, le comportement des fourmis, la gestion des phéromones, la simulation globale et l'affichage graphique. Cette organisation permet de séparer clairement les responsabilités, facilitant ainsi la maintenance, la lisibilité et l'évolution du projet.

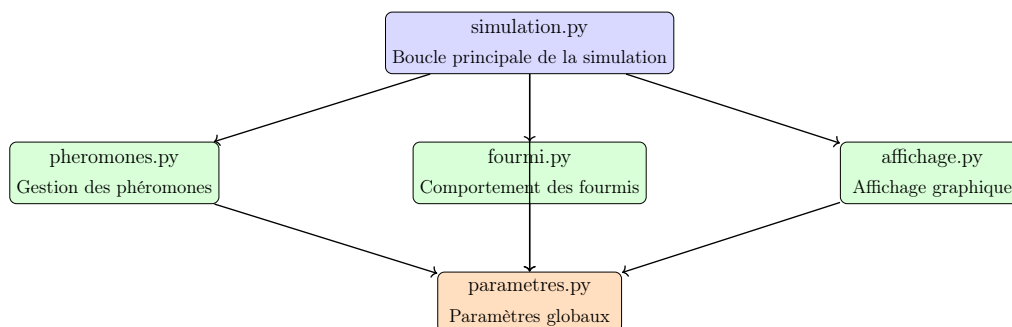


FIGURE 1 – Architecture modulaire du projet ACO

L'affichage graphique joue un rôle essentiel dans l'analyse du comportement de l'algorithme. Il permet d'observer visuellement les déplacements des fourmis, l'évolution des phéromones sur la grille et l'émergence progressive du chemin dominant reliant le nid à la nourriture.

Architecture logicielle du projet

Le fichier `simulation.py` constitue le cœur du programme : il orchestre la simulation, coordonne les interactions entre les fourmis, les phéromones et l'affichage, tout en s'appuyant sur les paramètres globaux définis dans `parametres.py`.

Paramètres de la simulation

Le fonctionnement de l'algorithme repose sur un ensemble de paramètres définissant à la fois l'environnement, le comportement des fourmis et l'évolution des phéromones. Ces

paramètres influencent directement la dynamique de la simulation et la convergence vers un chemin optimal.

5. Comportement des fourmis

Le comportement des fourmis dans la nature est particulièrement intéressant : chaque fourmi est un individu simple, doté de **capacités cognitives** limitées, mais l'ensemble de la colonie est capable de résoudre des problèmes complexes. Cette intelligence collective repose sur l'utilisation indirecte des phéromones, qui servent de moyen de communication entre les individus.

Le comportement d'une fourmi est composé de deux phases distinctes :

- une phase d'aller, durant laquelle la fourmi explore l'environnement à la recherche de la nourriture ;
- une phase de retour, durant laquelle elle revient vers le nid.

Au début de la simulation, les fourmis explorent aléatoirement la grille sans utiliser les phéromones. Elles choisissent leurs déplacements uniquement en fonction d'informations locales, notamment la distance à la nourriture et le coût des cases.

Après cette phase d'exploration initiale, les fourmis commencent à utiliser les phéromones pour guider leurs déplacements. Les cases contenant une quantité plus élevée de phéromones ont alors une probabilité plus importante d'être sélectionnées, ce qui favorise les chemins déjà empruntés.

Lorsqu'une fourmi atteint la nourriture, elle entame la phase de retour vers le nid en suivant exactement le chemin inverse. Pendant ce retour, elle dépose des phéromones sur les cases traversées, contribuant ainsi au renforcement collectif des chemins les plus efficaces.

6. Algorithme global de la simulation

La simulation débute par une phase d'initialisation au cours de laquelle la grille, les fourmis et les phéromones sont créées. Toutes les fourmis sont initialement placées au niveau du nid et aucune phéromone n'est présente dans l'environnement au début de la simulation.

À chaque itération de la simulation, plusieurs étapes successives sont réalisées :

- les phéromones présentes dans la grille s'évaporent légèrement ;
- les fourmis se déplacent en fonction de leur état (exploration ou retour) ;
- les fourmis en phase de retour déposent des phéromones sur les cases traversées ;
- un chemin dominant est calculé à partir de la répartition des phéromones afin de visualiser le résultat.

Au fil des itérations, la simulation permet d’observer l’apparition progressive d’un chemin stable reliant le nid à la nourriture. Ce chemin émerge naturellement des interactions répétées entre les fourmis et leur environnement.

Contrairement aux algorithmes déterministes, l’algorithme de colonie de fourmis présente plusieurs caractéristiques importantes :

- il est de nature probabiliste ;
- il ne garantit pas systématiquement l’obtention de la solution optimale ;
- il est robuste face aux changements de l’environnement.

7. Algorithme inspiré du comportement des fourmis

L’algorithme mis en œuvre dans ce projet s’inspire directement du comportement observé chez les fourmis dans la nature. Chaque fourmi agit individuellement selon des règles simples, sans connaissance globale de l’environnement. C’est l’interaction collective entre ces individus qui permet l’émergence progressive d’un comportement optimisé.

Phase 1 — Déplacement vers la nourriture (ALLER)

Étape 1 — Perception locale

À chaque instant, la fourmi ne dispose que d’une information locale. Elle observe uniquement :

- les cases voisines accessibles ;
- la présence éventuelle d’obstacles, qu’elle ne peut pas traverser ;
- elle ne possède aucune vision globale du chemin à parcourir.

Étape 2 — Exploration initiale

Lorsque la simulation se trouve encore dans la phase d’exploration globale, la fourmi adopte un comportement exploratoire :

1. elle choisit aléatoirement une case voisine accessible ;
2. ce choix privilégie les directions rapprochant de la nourriture ;
3. les zones difficiles sont pénalisées par un coût plus élevé ;
4. les phéromones ne sont pas prises en compte ;
5. le déplacement reste probabiliste et non déterministe.

Étape 3 — Exploitation progressive

Lorsque la phase d’exploration initiale est terminée, les fourmis commencent à exploiter l’information collective accumulée :

1. pour chaque case voisine, la fourmi évalue une attractivité locale basée sur :

- la quantité de phéromone présente ;
 - la proximité de la nourriture ;
 - le coût associé à la case ;
2. ces attractivités sont transformées en probabilités de déplacement ;
 3. la fourmi choisit ensuite sa prochaine case par tirage aléatoire selon ces probabilités.
- Ainsi, plus une case est utilisée collectivement, plus elle devient attractive, tout en conservant un caractère non strictement déterministe.

Étape 4 — Avancement

La fourmi se déplace vers la case choisie et mémorise cette position dans son chemin d'aller.

Étape 5 — Détection de la nourriture

Si la position courante correspond à la nourriture :

- la fourmi change d'état ;
- elle passe en phase de retour ;
- elle conserve en mémoire l'intégralité du chemin parcouru à l'aller.

Phase 2 — Retour vers le nid (RETOUR)

Étape 6 — Retour déterministe

Durant la phase de retour :

1. la fourmi suit exactement le chemin inverse de celui emprunté à l'aller ;
2. aucun choix n'est effectué :
 - aucun hasard ;
 - aucune exploration ;
3. la trajectoire de retour est strictement déterministe.

Étape 7 — Dépôt de phéromones

À chaque case traversée lors du retour :

1. la fourmi dépose une certaine quantité de phéromone ;
2. cette quantité est répartie sur l'ensemble du chemin ;
3. plus le chemin parcouru à l'aller est court, plus le dépôt de phéromone est important.

Ainsi, les chemins courts sont renforcés plus fortement que les chemins longs.

Étape 8 — Fin du cycle

Lorsque la fourmi atteint le nid :

- son chemin mémorisé est effacé ;
- son état repasse en phase d’aller ;
- elle repart immédiatement en exploration.

8. Expérimentations et résultats

Plusieurs scénarios ont été testés afin d’observer le comportement de l’algorithme de colonie de fourmis dans des environnements variés. Ces expérimentations ont permis d’analyser l’influence des obstacles, des zones difficiles et des paramètres de l’algorithme sur la dynamique de convergence.

Les résultats montrent que les fourmis convergent progressivement vers un chemin stable reliant le nid à la nourriture. Ce chemin n’apparaît pas immédiatement, mais émerge au fil des itérations grâce au mécanisme de dépôt et d’évaporation des phéromones.

L’augmentation du coût de certaines zones influence directement le chemin choisi par la colonie. Les zones difficiles sont naturellement évitées lorsque leur coût devient trop élevé, tandis que les obstacles obligent les fourmis à explorer des alternatives.

Dans l’ensemble, les résultats obtenus illustrent clairement le phénomène d’auto-organisation. Ils mettent en évidence la capacité de l’algorithme à faire émerger une solution collective efficace à partir de comportements individuels simples.

9. Analyse critique

L’approche par colonie de fourmis présente plusieurs avantages notables. Elle est robuste, adaptable et ne nécessite aucune connaissance globale de l’environnement. Grâce à son fonctionnement distribué, elle permet de gérer naturellement des environnements complexes, contraints ou dynamiques.

L’algorithme est également capable de s’adapter à des modifications de l’environnement, telles que l’ajout d’obstacles ou la modification des coûts, sans nécessiter une recombinaison complète de la solution. Cette propriété constitue un atout majeur par rapport aux approches déterministes classiques.

Cependant, cette méthode présente également certaines limites. La convergence vers une solution stable peut être lente et dépend fortement du choix des paramètres, tels que le taux d’évaporation ou l’importance accordée aux phéromones. De plus, l’algorithme ne garantit pas systématiquement l’obtention du chemin globalement optimal, en particulier dans des environnements complexes.

10. Conclusion générale

Ce projet a permis de mettre en évidence la puissance des algorithmes inspirés de la nature et, plus particulièrement, l'efficacité des algorithmes de colonie de fourmis pour résoudre des problèmes complexes de recherche de chemin. À travers la simulation développée, nous avons montré qu'il n'est pas nécessaire de disposer d'une intelligence centrale ou d'une connaissance globale de l'environnement pour faire émerger une solution pertinente. Des règles simples, appliquées localement par des agents autonomes, suffisent à produire une organisation collective cohérente.

L'algorithme étudié illustre parfaitement le rôle fondamental de l'organisation et de la communication indirecte. Les fourmis ne communiquent pas directement entre elles, mais à travers les phéromones qu'elles déposent dans l'environnement. Cette forme de communication distribuée permet à la colonie de partager une mémoire collective, d'exploiter les expériences passées et d'adapter progressivement son comportement. Ainsi, le chemin optimal n'est jamais imposé, mais construit étape par étape par l'ensemble du groupe.

Ce travail met également en lumière une réalité frappante : même des êtres extrêmement simples, tels que les fourmis, sont capables de résoudre des problèmes complexes lorsqu'ils agissent de manière organisée et coopérative. Cette observation, inspirée de la nature et de l'ordre établi par Dieu, rappelle que l'intelligence ne réside pas uniquement dans la complexité individuelle, mais aussi dans l'harmonie des interactions.

Au-delà de l'aspect algorithmique, ce projet invite à une réflexion plus large sur les systèmes collectifs, qu'ils soient naturels, sociaux ou technologiques. Il montre que l'organisation, la communication et la collaboration constituent des éléments clés pour résoudre des problèmes difficiles, que ce soit chez les fourmis, dans les sociétés humaines ou dans les systèmes informatiques modernes.

Enfin, ce projet constitue une base solide pour de futures améliorations, notamment l'adaptation dynamique des paramètres, la prise en compte d'environnements évolutifs ou l'extension à des problèmes plus complexes. Il démontre que s'inspirer de la nature est non seulement pertinent, mais aussi profondément riche d'enseignements, tant sur le plan scientifique que sur le plan humain.

11. Annexes

Les annexes contiennent les pseudo-codes détaillés, Le tableau des paramètres ainsi que des captures d'écran de la simulation permettant d'illustrer visuellement le fonctionnement de l'algorithme.

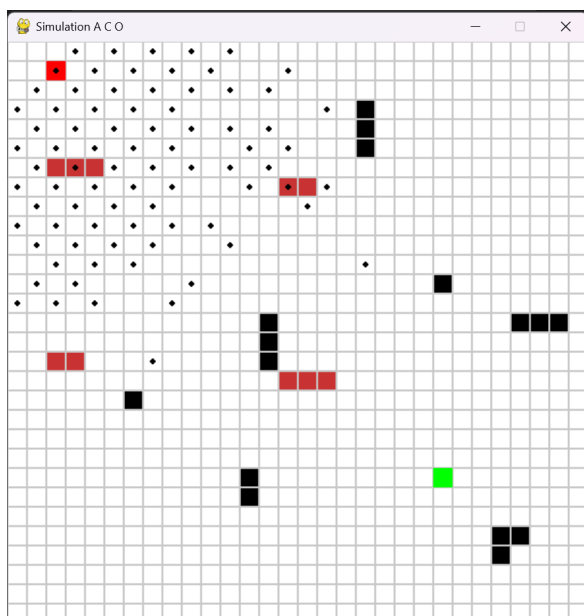


FIGURE 2 – État initial : environnement, obstacles et premières explorations

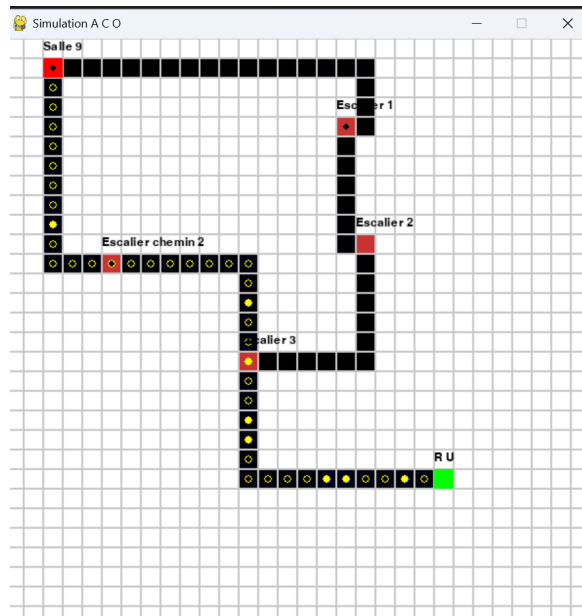


FIGURE 3 – Application au cas réel : chemins de la salle 9 au restaurant universitaire

Pseudo-code du comportement d'une fourmi

```

INITIALISER position ← NID
INITIALISER chemin_aller ← liste vide
INITIALISER etat ← ALLER

TANT QUE la simulation est active FAIRE
  SI etat = ALLER ALORS
    voisins ← cases voisines accessibles
    SI voisins n'est pas vide ALORS
      SI phase = EXPLORATION ALORS
        choisir une case voisine
        en fonction de la distance à la nourriture
        et du coût de la case
      SINON
        POUR chaque voisin FAIRE
          calculer attractivité locale
          en fonction de la phéromone
          de la distance à la nourriture
          et du coût de la case
        FIN POUR
        normaliser les attractivités
        choisir une case selon les probabilités
      FIN SI

    position ← case choisie
    ajouter position à chemin_aller

    SI position = NOURRITURE ALORS
      etat ← RETOUR
    FIN SI

  FIN SI

SINON
  position ← case précédente de chemin_aller
  déposer une quantité de phéromone sur position
  SI position est voisine du NID ALORS
    position ← NID
    chemin_aller ← liste vide
    etat ← ALLER
  FIN SI

FIN SI

FIN TANT QUE

```

Paramètre	Nom dans le programme	Description	Rôle dans l'algorithme
Nombre de lignes	NB_LIGNES	Nombre de lignes de la grille	Définit la hauteur de l'environnement
Nombre de colonnes	NB_COLONNES	Nombre de colonnes de la grille	Définit la largeur de l'environnement
Taille d'une case	TAILLE_CASE	Taille graphique d'une case	Utilisé uniquement pour l'affichage
Position du nid	POSITION_NID	Coordonnées du point de départ	Lieu initial de toutes les fourmis
Position de la nourriture	POSITION_NOURRITURE	Coordonnées de l'objectif	Point que les fourmis cherchent à atteindre
Nombre de fourmis	NB_FOURMIS	Nombre total de fourmis simulées	Influence la vitesse d'exploration et la convergence
Taux d'évaporation	EVAPORATION_PHEROMONE	Proportion de phéromone évaporée	Évite l'accumulation excessive de phéromones
Quantité de phéromone déposée	QUANTITE_PHEROMONE	Quantité déposée lors du retour	Renforce les chemins efficaces
Paramètre alpha	ALPHA	Importance des phéromones	Accentue l'exploitation des chemins existants
Paramètre beta	BETA	Importance de l'heuristique (distance)	Favorise les chemins proches de la nourriture
Nombre d'itérations d'exploration	NOMBRE_ITERATIONS_EXPL	Phase initiale sans phéromones	Garantit une exploration initiale suffisante
Coût des zones difficiles	COUT_ZONE_DIFFICILE	Coût supplémentaire d'une case	Décourage le passage par ces zones
Valeur maximale de phéromone	VALEUR_MAX_PHEROMONE	Limite supérieure de phéromone	Évite la domination excessive d'un chemin
Nombre d'images par seconde	NOMBRE_IMAGES_PAR_SECO	Fréquence de rafraîchissement	Influence la fluidité de la simulation

TABLE 1 – Paramètres utilisés dans la simulation ACO